

CHƯƠNG 3: NHÁNH VÀ GỘP

✔ Đã hoàn thành

Xung đột: khi hai thay đổi giẫm lên nhau

Sớm muộn nó cũng đến: bạn gõ `git merge`, và thay vì một dòng `Merge made by...` êm đẹp, Git dừng giữa chừng với chữ **CONFLICT** in hoa. Người mới thấy chữ đó thường nghĩ mình vừa làm hỏng repo. Chưa hỏng gì cả — bài này để bạn đọc được Git đang nói gì, và xử lý trong vài phút.

📖 Nội dung bài:

1. [Conflict không phải là lỗi](#)
2. [Tái hiện một conflict](#)
3. [Đọc marker xung đột](#)
4. [Giải conflict bằng tay](#)
5. [Rút lui: `git merge --abort`](#)
6. [Nỗi khổ riêng của dân data: notebook](#)
7. [Bảng tra lệnh](#)
8. [Lưu ý quan trọng](#)
9. [Thực hành trên máy bạn](#)

1. Conflict không phải là lỗi

Hai nhánh sửa hai chỗ khác nhau — kể cả trong cùng một file — thì Git tự trộn được, như bài trước bạn đã thấy. Conflict chỉ xảy ra khi hai nhánh sửa **cùng một dòng** theo hai cách khác nhau. Lúc đó giữ bản nào là quyết định **nghiệp vụ**, không phải bài toán thuật toán: ngưỡng lọc outlier nên là 5 hay 20 thì chỉ người hiểu dữ liệu mới trả lời được.

Nên hãy đọc **CONFLICT** đúng nghĩa của nó: **Git từ chối đoán**. Nó dừng lại, chỉ cho bạn đúng chỗ hai thay đổi giẫm lên nhau, và chờ bạn phân xử. Đó là hành vi đúng — một công cụ tự ý chọn hộ bạn mới là công cụ đáng sợ.

2. Tái hiện một conflict

Tự tạo một conflict để nhìn tận mắt. File `src/clean.py` đang có dòng `THRESHOLD = 10`. Trên một nhánh mới, hạ ngưỡng xuống 5:

```
git switch -c fix-threshold
```